

**HostelHub**

Complete Technical Guide & API Specs

Frontend, Backend, Database Schema, REST APIs & WebSockets Ka Pura Detailed Explanation (Hinglish Version)

PROJECT NAME

HostelHub Management System

DOCUMENT LANGUAGE

Hinglish (Hindi + English)

BACKEND ARCHITECTURE

Node.js, Express, Sequelize, PostgreSQL, Socket.io

FRONTEND ARCHITECTURE

Angular Standalone, TypeScript, Capacitor Android

LEAD DEVELOPER

Abhinav Kumar & HostelHub Engineering Team

DATE GENERATED

August 2026

Table of Contents (Hinglish)

1. System Intro & Architecture Overview (App Kaise Kam Karta Hai)
2. Tech Stack & Dependencies (Kaun Sa Tech Use Hua Hai)
3. Database Models & Schema Specifications (Database Tables Ka Structure)
 - 3.1 Users & Authentication Tables (User, PasswordResetOTP)
 - 3.2 Maintenance & Ticket Tables (Complaint)
 - 3.3 Mess Management Tables (MessMenu, MessSkip, MessFeedback)
 - 3.4 Attendance & Roll Call Tables (Attendance)
 - 3.5 Announcements & Chat Tables (Announcement, GroupChat, ChatMessage)
 - 3.6 System Settings & Notifications (Setting, Notification)
4. Complete REST API Endpoints Reference (Saare APIs Request & Response)
 - 4.1 Auth & User Profile APIs (/api/auth)
 - 4.2 User Management & Warden Approvals (/api/users)
 - 4.3 Maintenance Complaint Lifecycle APIs (/api/complaints)
 - 4.4 Mess Menu, Meal Skipping & Feedback APIs (/api/mess)
 - 4.5 Attendance Roll Call APIs (/api/attendance)
 - 4.6 Official Announcements APIs (/api/announcements)
 - 4.7 Group Chat APIs (/api/chat)
 - 4.8 Public Settings & Footer APIs (/api/settings)
 - 4.9 Live Notifications APIs (/api/notifications)
5. Socket.io WebSockets Guide (Real-time Live Events)
6. Special Business Rules & Security Guards (Re-approval, Back-Button, Image Fixes)

1. System Intro & Architecture Overview

HostelHub ek multi-portal digital hostel management app hai jo student complaints, daily roll call attendance, mess menu regulation, meal skipping, announcements, aur batch group chats ko streamline karta hai.

System Architecture: App me Decoupled Client-Server model implement hai. Frontend single-page application **Angular (Standalone Components)** me bana hai aur **Capacitor Bridge** ke dwara native Android APK me compile hota hai. Backend server **Node.js / Express.js** par chal raha hai jo **Sequelize ORM** ke through **PostgreSQL Database** se connect hota hai. Real-time updates ke liye **Socket.io WebSockets** ka upyog kiya gaya hai.

ROLE / USER TYPE	PORTAL ACCESS	MAIN FEATURES & PERMISSIONS
Student	Student Portal	Ticket raise karna (with photo), complaint status check karna, mess menu & snacks dekhna, meal skip karna, attendance metrics dekhna, hostel/batch group chat me message karna.
Warden	Warden Portal	Naye students aur critical edit requests ko approve/reject karna, staff ko ticket assign karna, daily roll call attendance mark karna, announcements post karna.
Staff	Staff Portal	Assigned jobs dekhna, work progress Update karna ("In Progress"), aur work completion photo proof submit karke job resolve karna.
Admin	Admin Portal	Warden/Staff accounts create karna, full user database audit karna, system settings aur footer/developer info manage karna.

2. Tech Stack & Dependencies

LAYER	TECHNOLOGY USED	KYU USE KIYA GAYA HAI (PURPOSE)
Frontend	Angular v19	Standalone Components, TypeScript, RxJS Observables for high speed SPA.
Mobile App Bridge	Capacitor v8	Android Native Build, Camera Plugin, App Preferences, Native Back Button Listener.
Styling & Design	Vanilla CSS Tokens	Modern Dark/Light Themes, Glassmorphism, Smooth Micro-animations, Vibrant Aesthetics.
Backend Runtime	Node.js & Express.js	High-performance REST API routing, Async Middleware stack.
Database & ORM	PostgreSQL & Sequelize	Structured Relational Tables, Foreign Key Relationships, Auto-indexing.
Real-Time Server	Socket.io	Live Bidirectional WebSockets for Chat, Notifications, Roll Call & Announcements.
Storage Service	Supabase / Local Disk Fallback	Cloud Bucket Image Storage with Base64 & Local Path fallback support.
Security & Auth	JWT & bcryptjs	JSON Web Token Auth Headers, Password Hashing, Role Guards.

3. Database Models & Schema Specifications

3.1 User Model (users table)

Student, Warden, Staff aur Admin ke saare account details is table me save hote hain.

```
User {
  id: INTEGER (PK, AutoIncrement)
  name: STRING (Full Name)
  email: STRING (Unique Email)
  password: STRING (Hashed Password)
  role: ENUM('student', 'warden', 'staff', 'admin') (Default: 'student')
  hostelBlock: STRING ('Boys Hostel 1', 'Boys Hostel 2', 'Girls Hostel 1', 'Girls Hostel 2')
  roomNumber: STRING (e.g. 'Room 222')
  rollNumber: STRING
  batch: STRING (e.g. 'Batch 2025')
  gender: STRING ('Male', 'Female', 'Other')
  phone: STRING
  bio: TEXT
  profilePicUrl: STRING
  status: ENUM('pending', 'active', 'rejected') (Default: 'pending')
  reApprovalStatus: BOOLEAN (Default: false) -- Gender/Batch edit hone par True hota hai
  googleId: STRING (Nullable)
}
```

3.2 Complaint Model (complaints table)

Maintenance tickets aur unka complete workflow lifecycle.

```
Complaint {
  id: INTEGER (PK, AutoIncrement)
  studentId: INTEGER (FK → users.id)
  staffId: INTEGER (FK → users.id, Nullable)
  title: STRING (Ticket Title)
  description: TEXT (Full Detail)
  category: ENUM('electrical', 'plumbing', 'carpentry', 'cleaning', 'other')
  priority: ENUM('low', 'medium', 'high', 'urgent') (Default: 'medium')
  status: ENUM('pending', 'assigned', 'in_progress', 'resolved')
  photoUrl: TEXT (Student Issue Image Attachment)
  completionPhotoUrl: TEXT (Staff Resolution Proof Image)
  feedbackRating: INTEGER (1 to 5 Stars)
  feedbackComment: TEXT
}
```

3.3 Mess Management Models

```
MessMenu {  
  id: INTEGER (PK, AutoIncrement)  
  dayOfWeek: ENUM('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')  
  breakfast: TEXT  
  lunch: TEXT  
  snacks: TEXT -- Evening Snacks Added  
  dinner: TEXT  
  isSpecial: BOOLEAN  
}  
  
MessSkip {  
  id: INTEGER (PK, AutoIncrement)  
  studentId: INTEGER (FK → users.id)  
  date: DATEONLY  
  mealType: ENUM('breakfast', 'lunch', 'snacks', 'dinner')  
}  
  
MessFeedback {  
  id: INTEGER (PK, AutoIncrement)  
  studentId: INTEGER (FK → users.id)  
  rating: INTEGER (1-5)  
  category: STRING ('Quality', 'Hygiene', 'Quantity', 'Behavior')  
  comment: TEXT  
}
```

3.4 Attendance Model (attendances table)

```
Attendance {  
  id: INTEGER (PK, AutoIncrement)  
  studentId: INTEGER (FK → users.id)  
  date: DATEONLY  
  status: ENUM('present', 'absent') -- Strictly Binary Statuses (Present & Absent)  
  markedBy: INTEGER (FK → users.id)  
  remarks: TEXT (Nullable)  
}
```

4. Complete REST API Endpoints Reference

4.1 Authentication & Profile APIs (/api/auth)

POST /api/auth/register

Description: Naya Student ya Warden account create karne ke liye API.

Request Body (JSON): { name, email, password, role, hostelBlock, roomNumber, rollNumber, batch, gender, phone }

Response (201 Created): { message: "Registration successful! Pending Warden approval.", user: { ... } }

POST /api/auth/login

Description: Credentials verify karke JWT Token return karta hai.

Request Body (JSON): { email, password }

Response (200 OK): { token: "eyJhbG...", user: { id, name, email, role, status } }

PUT /api/auth/profile

Description: Profile update API. Agar Student **Gender** ya **Batch** change karega, to automatic warning milne ke sath Warden re-approval flag (`reApprovalStatus = true`) activate ho jayega.

4.2 User Approvals APIs (/api/users)

GET /api/users/pending

Role: Warden, Admin

Description: Unapproved new registrations aur re-approval profile edits ki list deta hai.

PUT /api/users/approve/:userId

Role: Warden, Admin

Description: Pending student account ko active karta hai.

4.3 Maintenance Complaints APIs (/api/complaints)

POST /api/complaints/raise

Role: Student

Payload: multipart/form-data (title, description, category, priority, photo)

PUT /api/complaints/assign/:complaintId

Role: Warden

Body: { staffId: 3 }

Description: Warden staff member assign karta hai.

PUT `/api/complaints/update-status/:complaintId`

Role: Staff

Payload: multipart/form-data with status ('in_progress' or 'resolved') and completionPhoto.

4.4 Attendance APIs (/api/attendance)

POST `/api/attendance/mark`

Role: Warden, Admin

Body: `{ date: "2026-08-05", attendances: [{ studentId: 1, status: "present" }, { studentId: 2, status: "absent" }] }`

5. Socket.io WebSockets Guide (Real-time Live Events)

EVENT NAME	KAUN BHEJTA HAI	PAYLOAD DATA	KYU USE HOTA HAI (PURPOSE)
<code>join_room</code>	Client → Server	<code>"group_2"</code> ya <code>"user_10"</code>	Client specific group/user socket room join karta hai.
<code>send_message</code>	Client → Server	<code>{ groupId, senderId, message, attachmentUrl }</code>	Group me live chat message publish karta hai.
<code>new_message</code>	Server → Client	<code>{ id, groupId, message, sender: { ... } }</code>	Sabhi group members ke screen par instant message show karta hai.
<code>notification</code>	Server → Client	<code>{ message, type, createdAt }</code>	Live push/toast notification trigger karta hai.

6. Special Business Rules & Enforcements

1. Profile Re-approval Guard Rule: Student profile edit karte waqt Name, Phone, Bio, Room Number ya Profile Picture bina kisi warden approval ke instantly save ho jate hain. Par agar student **Gender** ya **Batch** change karta hai, to system warning popup show karta hai aur Warden Approval alert generate karta hai.

2. Roll Call Attendance Rule: Daily Attendance Roll Call se **Outing status remove** kar diya gaya hai. Ab Roll call strictly **Present** aur **Absent** me filter hoti hai, jise Warden Hostel Block wise ("Boys Hostel 1", "Girls Hostel 1", etc.) real-time track aur mark kar sakta hai.

3. Back-Button App Exit Modal: Android App par jab user back button press karta hai, to app sudden close hone ki jagah **Home tab par navigation** karti hai. Agar user dubara Back press karta hai to clean **Exit Confirmation Modal ("Yes, Exit / No, Cancel")** open hota hai.

HostelHub End-to-End Technical Documentation (Hinglish Edition) • Generated August 2026 • Confidential & Developer Reference